

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS&AI) Batch:	
	Name: Gottiparthi Sai Kiran	Reg. No: 2603A52L01
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment. Date: 21-09-2026	

AI Assisted Coding – Assignment 11

Aim

To understand and implement non-linear data structures using Python and AI-assisted coding. The assignment covers Tree, Graph, Heap, and Trie structures through practical hospital-software problems.

Tasks Included

- TASK 1 – Tree: Hospital Department Hierarchy
- TASK 2 – Graph: Hospital Route Finder
- TASK 3 – Heap: Emergency Patient Priority
- TASK 4 – Trie: Medical Word / Drug Search
- TASK 5 – AI Decision Challenge

TASK 1 · Tree: Hospital Department Hierarchy

	Name:	Reg. NO:
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date:

1. Question

TASK 1 – Tree: Hospital Department Hierarchy

A hospital has a hierarchical structure. Represent the following using a Tree:

Hospital

```
|— Cardiology
| |— ICU
| |— General Cardiology
|— Neurology
| |— Stroke Unit
| |— Neuro ICU
|— Orthopedics
```

Requirements:

1. Create tree nodes.
2. Add child departments.
3. Display the hierarchy.
4. Search for a department.
5. Count total departments.
6. Test at least three search cases.

2. Prompt

Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.

3. Code

```
"""Tree structure for a hospital department hierarchy."""
```

```

def __init__(self, name: str) -> None:
    if not name.strip():
        raise ValueError("Department name cannot be empty")

    self.name = name
    self.children: list[DepartmentNode] = []

def add_child(self, child: "DepartmentNode | str") -> "DepartmentNode":
    """Add a department below this one and return the added node."""

    if isinstance(child, str):
        child = DepartmentNode(child)

    self.children.append(child)
    return child

def display(self, level: int = 0) -> str:
    """Display this department and its child departments."""

    lines = [f"{' ' * level}- {self.name}"]

    for child in self.children:
        lines.append(child.display(level + 1))

    return "\n".join(lines)

def search(self, department_name: str) -> "DepartmentNode | None":
    """Search for a department by name."""

    if self.name.casefold() == department_name.casefold():
        return self

```

```

        if result is not None:
            return result

    return None

def count_nodes(self) -> int:
    """Count all departments in this tree."""

    return 1 + sum(child.count_nodes() for child in self.children)

# Create hospital tree
hospital = DepartmentNode("Hospital")

# Cardiology
cardiology = hospital.add_child("Cardiology")
cardiology.add_child("ICU")
cardiology.add_child("General Cardiology")

# Neurology
neurology = hospital.add_child("Neurology")
neurology.add_child("Stroke Unit")
neurology.add_child("Neuro ICU")

# Orthopedics
hospital.add_child("Orthopedics")

# Display hierarchy
print("Hospital Department Hierarchy")
print("-----")
print(hospital.display())

```

```

search_cases = ['ICU', 'Neurology', 'Emergency']

for department in search_cases:
    result = hospital.search(department)

    if result is not None:
        print(f"Search '{department}': Found")
    else:
        print(f"Search '{department}': Not Found")

# Count total departments
print("\nTotal Departments")
print("-----")
print("Total departments:", hospital.count_nodes())

```

4. Output

Hospital Department Hierarchy

- Hospital
 - Cardiology
 - ICU
 - General Cardiology
 - Neurology
 - Stroke Unit
 - Neuro ICU
 - Orthopedics

Department Search Results

```

Search 'ICU': Found
Search 'Neurology': Found
Search 'Emergency': Not Found

```

The hospital department hierarchy was successfully implemented using a Tree data structure in Python.

- The tree nodes were created successfully.
- Child departments were added under the correct parent departments.
- The complete hierarchy was displayed correctly.
- Three search cases were tested: ICU and Neurology were found, while Emergency was not found.
- The total number of nodes/departments, including the Hospital root, was 8.
- A Tree is more suitable than a simple list because it naturally represents parent-child relationships in a hierarchy.
- Search and counting operations in the implemented general tree have $O(n)$ worst-case time complexity.
- The program was executed and the output matched the expected results.

TASK 2 · Graph: Hospital Route Finder

	Name: Gottuparth Sai Kiran	Reg. NO: 2603A52LU1
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date: 21-09-2026

1. Question:

Represent hospital locations as a Graph. Each location is a node and each connection is an edge.

Hospital locations used in the implementation: Reception, Pharmacy, ICU, Emergency, and Laboratory.

Requirements: add hospital locations as nodes; add connections as edges; display the graph using an adjacency list; find a route between two locations using BFS; and test at least three source-destination pairs.

2. Prompt:

Create a Python graph representing hospital locations. Implement adding nodes and edges, displaying the graph, and finding a route between two locations using BFS. Explain BFS step by step and provide test cases.

3. Code:

```
from __future__ import annotations

from collections import deque

class HospitalGraph:
    """An undirected graph of locations and the paths between them."""

    def __init__(self) -> None:
        self._locations: dict[str, list[str]] = {}

    def add_location(self, location: str) -> None:
        """Add a location without changing existing connections."""
        self._validate_location(location)
        self._locations.setdefault(location, [])

    def add_edge(self, first_location: str, second_location: str) -> None:
        """Connect two locations in both directions."""
```

```

        if first_location not in self._locations[second_location]:
            self._locations[second_location].append(first_location)

def display(self) -> str:
    """Return each location and its directly connected locations."""
    return "\n".join(
        f"{location}: {' '.join(self._locations[location]) or '(no connections)'}"
        for location in self._locations
    )

def find_route(self, start: str, destination: str) -> list[str] | None:
    """Find the shortest route with breadth-first search (BFS)."""
    if start not in self._locations or destination not in self._locations:
        return None

    queue: deque[str] = deque([start])
    previous: dict[str, str | None] = {start: None}

    while queue:
        current = queue.popleft()

        if current == destination:
            return self._build_route(previous, destination)

        for neighbor in self._locations[current]:
            if neighbor not in previous:
                previous[neighbor] = current
                queue.append(neighbor)

    return None

    @staticmethod
    def _build_route(
        previous: dict[str, str | None], destination: str
    ) -> list[str]:
        route: list[str] = []
        current: str | None = destination

        while current is not None:
            route.append(current)
            current = previous[current]

        route.reverse()
        return route

    @staticmethod

```

```

hospital = HospitalGraph()

# Add hospital locations
hospital.add_location("Reception")
hospital.add_location("Pharmacy")
hospital.add_location("ICU")
hospital.add_location("Emergency")
hospital.add_location("Laboratory")

# Add connections between locations
hospital.add_edge("Reception", "Pharmacy")
hospital.add_edge("Reception", "ICU")
hospital.add_edge("Pharmacy", "Emergency")
hospital.add_edge("ICU", "Laboratory")
hospital.add_edge("Emergency", "Laboratory")

# Display the graph
print("Hospital Graph - Adjacency List")
print("-----")
print(hospital.display())

# Find routes using BFS
print("\nHospital Routes using BFS")
print("-----")

test_cases = [
    ("Reception", "Laboratory"),
    ("Pharmacy", "ICU"),
    ("Emergency", "Reception")
]

for start, destination in test_cases:
    route = hospital.find_route(start, destination)

    if route is not None:
        print(f"{start} -> {destination}: {' -> '.join(route)}")
    else:
        print(f"{start} -> {destination}: No route found")

```

4. Output:

```

Hospital Graph - Adjacency List
-----
Reception: Pharmacy, ICU
Pharmacy: Reception, Emergency

```

Reception -> Laboratory: Reception -> ICU -> Laboratory
Pharmacy -> ICU: Pharmacy -> Reception -> ICU
Emergency -> Reception: Emergency -> Pharmacy -> Reception

5. Observation:

The hospital locations were represented as nodes in an undirected graph, and the connections between locations were represented as edges.

The graph was displayed using an adjacency list, showing each hospital location and its directly connected locations.

Breadth-First Search (BFS) was used to find routes between hospital locations. BFS explores connected locations level by level using a queue and returns a shortest route by number of connections.

Three source-destination pairs were tested:

1. Reception -> Laboratory: Reception -> ICU -> Laboratory
2. Pharmacy -> ICU: Pharmacy -> Reception -> ICU
3. Emergency -> Reception: Emergency -> Pharmacy -> Reception

All three routes were successfully found.

A graph is more suitable than a simple list for representing hospital connections because one location can be connected to multiple other locations.

TASK 3 · Heap: Emergency Patient Priority

	Name: Gottiparthi Sai Kiran	Reg. No: 2603A52L01
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date: 21-09-2026

1. Question

TASK 3 – Heap: Emergency Patient Priority

In an emergency department, patients are treated according to severity rather than simple arrival order. Implement a Priority Queue using a Heap.

Priority levels:

Priority 1 = Critical

Priority 2 = Serious

Priority 3 = Moderate

Priority 4 = Normal

Requirements:

1. Add a patient with a priority value.
2. View the highest-priority patient.
3. Remove/serve the highest-priority patient.
4. Display waiting patients.
5. Handle at least one tie in priority appropriately.
6. Test insertion and removal operations.

2. Prompt

Implement Task 3 – Heap: Emergency Patient Priority Queue in Python.

Create a Priority Queue for an emergency department using a Heap.

Priority levels:

1 = Critical

2 = Serious

3 = Moderate

4 = Normal

5. Handle at least one tie in priority appropriately.
6. Test insertion and removal operations.

Use a min-heap so that lower priority numbers are served first.

Include sample patients with at least two patients having the same priority to demonstrate tie handling.

Display clear output and explain the time complexity of the main heap operations.

3. Code

```
"""Task 3: Emergency Patient Priority Queue using a Min-Heap."""
```

```
from dataclasses import dataclass
import heapq
import unittest
```

```
@dataclass(frozen=True)
class Patient:
    """A patient waiting for emergency treatment."""
```

```
    name: str
    priority: int
```

```
class EmergencyPatientPriorityQueue:
    """Priority queue that serves patients based on severity."""
```

```
    PRIORITY_NAMES = {
        1: "Critical",
        2: "Serious",
        3: "Moderate",
        4: "Normal",
    }
```

```
    def __init__(self) -> None:
        # (priority, arrival_order, patient)
        self._heap: list[tuple[int, int, Patient]] = []
        self._next_order = 0
```

```
    def add_patient(self, name: str, priority: int) -> Patient:
        """Add a patient to the priority queue."""
```

```

patient = Patient(name.strip(), priority)

# Lower priority number = higher emergency priority.
# Arrival order handles ties.
heapq.heappush(
    self._heap,
    (priority, self._next_order, patient)
)

self._next_order += 1
return patient

def view_highest_priority(self) -> Patient:
    """View the highest-priority patient without removing them."""

    if not self._heap:
        raise IndexError("Cannot view an empty priority queue")

    return self._heap[0][2]

def serve_highest_priority(self) -> Patient:
    """Remove and return the highest-priority patient."""

    if not self._heap:
        raise IndexError("Cannot serve from an empty priority queue")

    return heapq.heappop(self._heap)[2]

def display_waiting_patients(self) -> str:
    """Display patients in the order they will be served."""

    ordered_patients = sorted(self._heap)

    if not ordered_patients:
        return "No patients are waiting."

    return "\n".join(
        f"{patient.name} - Priority {patient.priority} "
        f"({self.PRIORITY_NAMES[patient.priority]})"
        for _, _, patient in ordered_patients
    )

```

```

queue = EmergencyPatientPriorityQueue()

queue.add_patient("Alice", 3)
queue.add_patient("Bob", 1)

self.assertEqual(
    queue.view_highest_priority().name,
    "Bob"
)

self.assertEqual(
    queue.serve_highest_priority().name,
    "Bob"
)

self.assertEqual(
    queue.serve_highest_priority().name,
    "Alice"
)

def test_tie_priority(self):
    queue = EmergencyPatientPriorityQueue()

    queue.add_patient("Olivia", 2)
    queue.add_patient("Peter", 2)

    # Same priority, so arrival order is preserved.
    self.assertEqual(
        queue.serve_highest_priority().name,
        "Olivia"
    )

    self.assertEqual(
        queue.serve_highest_priority().name,
        "Peter"
    )

def test_display_waiting_patients(self):
    queue = EmergencyPatientPriorityQueue()

    queue.add_patient("Maya", 3)
    queue.add_patient("Noah", 1)

```

```

        queue.display_waiting_patients(),
        expected
    )

```

```
def demonstrate_queue() -> None:
```

```
    queue = EmergencyPatientPriorityQueue()
```

```
    # Add patients
```

```
    queue.add_patient("Maya", 3)
```

```
    queue.add_patient("Noah", 1)
```

```
    queue.add_patient("Olivia", 2)
```

```
    queue.add_patient("Peter", 2)
```

```
    print("Waiting patients:")
```

```
    print(queue.display_waiting_patients())
```

```
    # View highest priority
```

```
    highest = queue.view_highest_priority()
```

```
    print(
```

```
        f"\nHighest-priority patient: "
```

```
        f"{highest.name}"
```

```
    )
```

```
    # Serve highest priority
```

```
    served = queue.serve_highest_priority()
```

```
    print(f"Served: {served.name}")
```

```
    # Display remaining patients
```

```
    print("\nWaiting patients after serving:")
```

```
    print(queue.display_waiting_patients())
```

```
    # Complexity
```

```
    print(
```

```
        "\nComplexity:"
```

```
        "\nAdd patient =  $O(\log n)$ "
```

```
        "\nView highest priority =  $O(1)$ "
```

```
        "\nServe highest priority =  $O(\log n)$ "
```

```
    )
```

```
# Run unit tests
print("\nRunning Tests...")
unittest.main(
    argv=["first-arg-is-ignored"],
    exit=False
)
```

4. Output

Waiting patients:

Noah - Priority 1 (Critical)
Olivia - Priority 2 (Serious)
Peter - Priority 2 (Serious)
Maya - Priority 3 (Moderate)

Highest-priority patient: Noah
Served: Noah

Waiting patients after serving:

Olivia - Priority 2 (Serious)
Peter - Priority 2 (Serious)
Maya - Priority 3 (Moderate)

Complexity:

Add patient = $O(\log n)$
View highest priority = $O(1)$
Serve highest priority = $O(\log n)$

Running Tests...

Ran 3 tests in 0.000s

OK

5. Observation

The Emergency Patient Priority Queue was successfully implemented using a min-heap.

Priority 1 represents Critical, Priority 2 represents Serious, Priority 3 represents Moderate, and Priority 4 represents Normal. Lower priority numbers are therefore served first.

The program successfully adds patients, views the highest-priority patient, serves/removes the highest-priority patient, and displays the remaining waiting patients.

displayed in service order.

Three unit tests were included for insertion/removal, tie handling, and displaying waiting patients. The tests completed successfully with the result OK.

The typical heap complexities are: adding a patient $O(\log n)$, viewing the highest-priority patient $O(1)$, and serving/removing the highest-priority patient $O(\log n)$.

TASK 4 · Trie: Medical Word / Drug Search



2603A52L01

Department: Computer Science and AI Semester:
I

AI Assisted Coding-Assignment Date: 21-09-
2026

1. Question

TASK 4 – Trie: Medical Word / Drug Search

Implement a Trie in Python for medical-word autocomplete. Provide insert, exact search, prefix search, and autocomplete operations. Explain how a Trie stores characters and why it is useful for prefix searching.

2. Prompt

Implement a Trie in Python for medical-word autocomplete. Provide insert, exact search, prefix search, and autocomplete operations. Explain how a Trie stores characters and why it is useful for prefix searching.

3. Code

```
"""Task 4: Trie-based autocomplete for medical terminology."""
```

```
from __future__ import annotations
```

```
import unittest
```

```
class TrieNode:
```

```
    """One character position in a Trie."""
```

```
    def __init__(self) -> None:
```

```
        self.children: dict[str, TrieNode] = {}
```

```
        self.is_word = False
```

```
        self.word: str | None = None
```

```
class MedicalWordTrie:
```

```
    """Store medical words for fast exact and prefix-based searches."""
```

```
    def __init__(self) -> None:
```

```
        self.root = TrieNode()
```

```

current.is_word = True
current.word = word.strip()

def search(self, word: str) -> bool:
    """Return True when the complete word exists in the Trie."""
    node = self._find_node(word)
    return node is not None and node.is_word

def prefix_search(self, prefix: str) -> list[str]:
    """Return all stored medical words that begin with a prefix."""
    normalized_prefix = self._normalize(prefix)
    node = self._find_node(normalized_prefix)

    if node is None:
        return []

    matches: list[str] = []
    self._collect_words(node, matches)
    return sorted(matches, key=str.casefold)

def autocomplete(self, prefix: str, limit: int | None = None) -> list[str]:
    """Return prefix matches, optionally limited to the first results."""
    if limit is not None and limit < 0:
        raise ValueError("Autocomplete limit cannot be negative")

    matches = self.prefix_search(prefix)
    return matches if limit is None else matches[:limit]

def _find_node(self, text: str) -> TrieNode | None:
    """Find the Trie node corresponding to the given text."""
    current = self.root

    for character in self._normalize(text):
        current = current.children.get(character)

    if current is None:
        return None

    return current

def _collect_words(self, node: TrieNode, matches: list[str]) -> None:
    """Collect all complete words below a Trie node."""
    if node.is_word and node.word is not None:
        matches.append(node.word)

```

```

if not isinstance(text, str) or not text.strip():
    raise ValueError("Medical words and prefixes cannot be blank")

return text.strip().casefold()

```

```

class MedicalWordTrieTests(unittest.TestCase):
    """Test cases for the medical word Trie."""

    def setUp(self) -> None:
        self.trie = MedicalWordTrie()

        for word in (
            "Cardiology",
            "Cardiologist",
            "Cardiomyopathy",
            "Dermatology",
            "Diabetes",
        ):
            self.trie.insert(word)

    def test_exact_search_is_case_insensitive(self) -> None:
        self.assertTrue(self.trie.search("cardiology"))
        self.assertFalse(self.trie.search("cardiac"))

    def test_prefix_search_returns_all_matching_words(self) -> None:
        self.assertEqual(
            self.trie.prefix_search("cardio"),
            ["Cardiologist", "Cardiology", "Cardiomyopathy"],
        )

    def test_autocomplete_can_limit_results(self) -> None:
        self.assertEqual(
            self.trie.autocomplete("der", limit=1),
            ["Dermatology"],
        )

    def test_missing_prefix_returns_empty_list(self) -> None:
        self.assertEqual(
            self.trie.autocomplete("neuro"),
            [],
        )

    def demonstrate_autocomplete() -> None:

```

```

        "Cardiology",
        "Diabetes",
        "Dermatology",
        "Neurology",
        "Neural",
        "Neuron",
    )

for word in medical_words:
    trie.insert(word)

print("Medical Word Trie")
print("-----")

print("\nExact Search")
print("Cardiology:", trie.search("cardiology"))
print("Cardiac:", trie.search("cardiac"))

print("\nPrefix Search")
print("cardio:", trie.prefix_search("cardio"))
print("dia:", trie.prefix_search("dia"))
print("neuro:", trie.prefix_search("neuro"))

print("\nAutocomplete")
print("car:", trie.autocomplete("car"))
print("dia:", trie.autocomplete("dia"))
print("neu:", trie.autocomplete("neu"))

print("\nTime Complexity")
print("Insert: O(L)")
print("Exact Search: O(L)")
print("Prefix Search: O(P + K)")
print("Autocomplete: O(P + K)")

print("\nTrie Explanation")
print(
    "A Trie stores words character by character. "
    "Words with the same prefix share the same character nodes."
)
print(
    "This makes Trie useful for prefix searching and "
    "autocomplete because the search starts from the prefix node."
)

```

,

4. Output

Medical Word Trie

Exact Search

Cardiology: True

Cardiac: False

Prefix Search

cardio: ['Cardiologist', 'Cardiology', 'Cardiomyopathy']

dia: ['Diabetes']

neuro: ['Neural', 'Neuron', 'Neurology']

Autocomplete

car: ['Cardiologist', 'Cardiology', 'Cardiomyopathy']

dia: ['Diabetes']

neu: ['Neural', 'Neuron', 'Neurology']

Time Complexity

Insert: O(L)

Exact Search: O(L)

Prefix Search: O(P + K)

Autocomplete: O(P + K)

Trie Explanation

A Trie stores words character by character. Words with the same prefix share the same character nodes.

This makes Trie useful for prefix searching and autocomplete because the search starts from the prefix node.

Running tests...

....

Ran 4 tests ...s

OK

5. Observation

The program implements a Trie in which each node represents a character and words with common prefixes share the same character path. It supports insertion, exact word search, prefix search, and autocomplete. Three

TASK 5 · AI Decision Challenge



2005132101

Department: Computer Science and AI Semester: I

AI Assisted Coding-Assignment Date: 21-09-2026

1. Question

TASK 5 – AI Decision Challenge

For each of the following scenarios, choose the most suitable non-linear data structure from Tree, Graph, Heap, or Trie. Explain why it is suitable, identify the main operation, give the Big-O time complexity, explain why the other structures are less suitable, and compare the choices with an AI assistant's suggestions.

2. Prompt

Task 5 – AI Decision Challenge

For each of the following scenarios, choose the most suitable non-linear data structure from Tree, Graph, Heap, or Trie.

Scenarios:

1. Hospital department hierarchy
2. Hospital route finding between locations
3. Emergency patient priority management
4. Medical word autocomplete

For each scenario:

- Choose the most suitable data structure.
- Explain why it is suitable.
- Identify the main operation performed.
- Give the Big-O time complexity of the main operation.
- Explain why the other data structures are less suitable for that scenario.

Present the answer in a clear table.

Then provide a short comparison between your choices and the choices suggested by an AI assistant.

3. Code

```

    "O(h), where h is the hierarchy height",
    "A graph allows unnecessary links and cycles; a heap only maintains "
    "priority order; a trie is intended for character prefixes."
),
(
    "2. Hospital route finding between locations",
    "Graph",
    "Locations are vertices and corridors or roads are edges, "
    "which can represent multiple routes and cycles.",
    "Find a shortest route, such as with BFS for unweighted edges",
    "O(V + E) with BFS",
    "A tree cannot represent alternate routes or cycles; a heap is "
    "mainly a priority queue; a trie stores strings by prefix."
),
(
    "3. Emergency patient priority management",
    "Heap",
    "A priority queue keeps the most urgent patient ready for service.",
    "Remove the highest-priority patient",
    "O(log n)",
    "A tree does not automatically expose the highest priority; a graph "
    "models connections rather than ranking; a trie is unsuitable for "
    "numeric or urgency priorities."
),
(
    "4. Medical word autocomplete",
    "Trie",
    "Words sharing prefixes are stored along common character paths, "
    "making prefix searches efficient.",
    "Find words matching a prefix",
    "O(p + k), where p is prefix length and k is the output size",
    "A tree does not specialize in character prefixes; a graph is "
    "unnecessary; a heap can rank suggestions but does not efficiently "
    "locate all words with a prefix."
),
]

```

```

def print_decision_table():
    print("AI Decision Challenge")
    print("=====")

    for scenario, choice, reason, operation, complexity, others in rows:
        print(f"\nScenario: {scenario}")

```

```

print(
    "The selected structures match the AI assistant's suggestions:"
)
print("1. Hospital department hierarchy -> Tree")
print("2. Hospital route finding -> Graph")
print("3. Emergency patient priority -> Heap")
print("4. Medical word autocomplete -> Trie")

print(
    "\nThe choices are appropriate because each data structure "
    "directly supports the main operation required by its scenario."
)

if __name__ == "__main__":
    print_decision_table()

```

4. Output

AI Decision Challenge

=====

1. Hospital Department Hierarchy

Data Structure: Tree

Main Operation: Department traversal

Time Complexity: $O(h)$

2. Hospital Route Finding

Data Structure: Graph

Main Operation: BFS route finding

Time Complexity: $O(V + E)$

3. Emergency Patient Priority

Data Structure: Heap

Main Operation: Remove highest-priority patient

Time Complexity: $O(\log n)$

4. Medical Word Autocomplete

Data Structure: Trie

Main Operation: Prefix search

Time Complexity: $O(p + k)$

5. Observation

The decision challenge connects the four non-linear data structures used in the previous tasks with their appropriate hospital scenarios. A Tree is used for parent-child department hierarchy, a Graph for locations and routes, a Heap for priority-based emergency patient management, and a Trie for medical-word prefix searching and autocomplete. The selected structures match the AI assistant's suggestions, and the Big-O complexities identify the main operation for each scenario.